Strangler Pattern: A Detailed Example

Let's work on a Java code example that puts the Strangler Pattern into action.

Setting the Stage: The Legacy System

First, let's set the stage with a simple example of a legacy system. Imagine a basic class, `LegacySystem`, which has methods for `createAccount()`, `deposit()`, `withdraw()`, and `checkBalance()`. Here's how this class might look:

```
public class LegacySystem {
    public void createAccount(String name) {
        // ... Old logic for creating an account ...
    }
    public void deposit(int amount) {
        // ... Old logic for depositing money ...
    }
    public void withdraw(int amount) {
        // ... Old logic for withdrawing money ...
    }
    public int checkBalance() {
        // ... Old logic for checking balance ...
        return 0; // placeholder
    }
}
```

Creating the New System

Next, let's introduce our new system, appropriately named `NewSystem`. It will have the same methods but with updated logic:

```
public class NewSystem {
    public void createAccount(String name) {
        // ... New logic for creating an account ...
    }
    public void deposit(int amount) {
        // ... New logic for depositing money ...
    }
    public void withdraw(int amount) {
        // ... New logic for withdrawing money ...
    }
    public int checkBalance() {
        // ... New logic for checking balance ...
        return 0; // placeholder
    }
}
```

The Facade Interface

Now, it's time to bring in our star player, the Facade Interface. This is the class that will be used by the outside world to interact with our system. It has an instance of both the legacy and the new system, and directs the requests accordingly.

At first, all requests are sent to the legacy system. But as we implement new functionality in the new system, the Facade Interface starts redirecting those requests to the new system.

Here's what our `BankingFacade` could look like:

```
public class BankingFacade {
    private LegacySystem legacySystem;
    private NewSystem newSystem;

    public BankingFacade(LegacySystem legacySystem, NewSystem newSystem) {
        this.legacySystem = legacySystem;
        this.newSystem = newSystem;
    }

    public void createAccount(String name) {
        if(newSystemReadyFor("createAccount")){
            newSystem.createAccount(name);
        } else {
            legacySystem.createAccount(name);
        }
    }

    public void deposit(int amount) {
        if(newSystemReadyFor("deposit")){
            newSystem.deposit(amount);
        } else {
            legacySystem.deposit(amount);
        }
    }

    public void withdraw(int amount) {
        if(newSystemReadyFor("withdraw")){
            newSystem.withdraw(amount);
        } else {
            legacySystem.withdraw(amount);
        }
    }

    public int checkBalance() {
        if(newSystemReadyFor("checkBalance")){
            return newSystem.checkBalance();
        } else {
            return legacySystem.checkBalance();
        }
    }

    private boolean newSystemReadyFor(String functionality) {
        // ... Check if new system is ready for given functionality ...
        return false; // placeholder
    }
}
```

The `BankingFacade` class directs the flow of operations between the old and new systems.

The Gradual Transition: Replacing Functionality

Now, let's assume that we've replaced the `createAccount` functionality in the new system. We update the `newSystemReadyFor` method to return true for the `createAccount` functionality. And with that all `createAccount` requests are now directed to the new system.

As we replace more functionalities, we keep updating the `newSystemReadyFor` method, and gradually, the new system takes over.

← Previous
The Architecture Of The Strangler Pattern

Next →
Key Insights And Implications

☐ Mark as Completed